Breakpoint

Find where a walking robot's policy breaks, harden it, and know the odds before you ship.

github.com/AyanBhardwaj1/RL-Robot
Python · C++17 · JavaScript · MuJoCo

84.1 readiness of the best quadruped policy, up from 66.8 for its bare gait	**0 / 60** teleop worlds it fell in after a latency gate (10% before)	**44 vs 120** N·s side shove survived: walking biped vs wheeled base	**3.6×** smaller INT8 policy with no measured robustness loss

What it does

- **Breaking points.** Bisects 17 stresses (shoves at 4 gait phases, slope, stairs, friction, payload, weak motors, latency, noise) to the exact edge the policy survives.
- **Failure hunting.** Cross-entropy search for the mildest failing worlds, a MAP-Elites atlas of failure types, and shrinking to minimal JSON regression tests.
- **Hardening.** PPO, written from scratch in numpy, replays failures and target deployment worlds; a checkpoint ships only if it beats the start.
- **Deployment odds.** Failure rate per site profile with a 95% upper bound and causes; a hazard log turns them into risk ratings with verify commands.
- **Humanoid lab.** Capture-point push recovery for a 72 kg biped, a wheeled-base comparison, and odds by job, in Python, a browser port and a C++17 port held equal by tests.

What it found

- A policy that walked in noisy training stood still when run for real, and scored below the gait it was built on.
- A second hardening round overfit; a fresh audit caught it.
- Training could not remove latency failures; gating the learned residual on measured link latency did.
- Legs vs wheels: the wheeled base survives 1.5× bigger shoves from the front and 2.7× from the side, but fails 45% of home-care routes (stairs); the biped fails 0% there, but 17% of trailer shifts.
- Design levers, paired on 750 shifts: a 15% faster swing saves 25 and loses none ($p < 0.001$); +25% ankle torque saves none.
- A capturability map and an independent review exposed three modelling errors (a non-causal latency model, a swing foot that still held weight, ankles that ignored lag); each fix made the biped's odds worse and more honest.
- The C++ controller step costs under 1 µs; the worst stock-kernel wake-up ranged from 2 to 15 ms over five runs, which is the case for PREEMPT_RT.

Where it meets each team

Team	Evidence in the repo
Controls	LIP + capture-point stepping with ankle, friction, reach, swing and stance-leg limits, Smith-predictor latency compensation (`humanoid.py`, `cpp/capture_point.hpp`)
Simulation	Procedural MJCF quadruped and terrain, validated scenario schema, physical convention tests
Robot learning	numpy PPO with finite-difference-checked backprop, failure replay, deterministic checkpoint selection
ML systems	FP16 and per-channel INT8 compression judged by breaking points, not only action error
Real-time / embedded	Allocation-free 2 ms C++ step, 500 Hz loop benchmark, latency and actuator-strength budgets
Platform / test	Over 90 tests, CI with `-Werror` C++ build, regression suites, a certify gate with hash and versions
Fleet / safety	Stateless HTTP API, site profiles with confidence bounds, measured hazard log with mitigations

Built with an AI coding agent and checked by four independent code reviews (31 findings fixed with regression tests). The robots are generic, not models of any product; numbers compare designs, not certify hardware.